

RTB Course Project requirements

1 What is required?

As a final project, you need to provide a complete description of your project in PDF format according to the requirements described below, as well as an MVP demonstrating your project.

2 Detailed description of the product problem

2.1 What we want to see

For your project, you should define:

- Target audience and needs
- Functional boundaries of the service (Which functions does the service cover, and which does it not?)
- Key performance metrics

2.2 Example: URL shortener

- Audience: Marketers (70%), bloggers (25%), corporate clients (5%)
- Needs:
 - Shortening long links for social media
 - Tracking clicks without complex tools (Google Analytics)
 - Not covered: User accounts (It turned out that users weren't interested in anonymous usage).
- Metrics:

Metric	Calculation	Expectation
DAU	Unique sessions/day	1,000
Retention	Percentage returning after 7	30%

	days	
Avg. clicks	Clicks per user	5.2

3 Converting product metrics into technical parameters

3.1 What we want to see

You should define the important technical parameters for your product (explain why they are important) and come up with rules/formulas (explain why they are set this way) for how you determine them. Then convert the technical parameters according to these requirements.

3.2 Example calculations

- **RPS:** RPS is very important to us because we want to understand how important it is to have many servers, etc.

$$\frac{\text{Avg. clicks} \cdot \text{DAU}}{\text{seconds}_{\text{day}}} \cdot \text{busy_factor} = \frac{5.2 \cdot 1000}{86400} \cdot 10 = 0.6 \rightarrow \text{expect 0.6 RPS}$$
- **Storage:** It's important for us to understand how much information we store to decide in advance how much memory to allocate to the database.
 - We need to store links and where they lead, with a maximum of 7 characters in a short link (7 bytes), say 100 characters in a long link (100 bytes), we also want to store when the record was created (time - 12 bytes) and when the last click occurred (time - 12 bytes), totaling (7 + 100 + 12 + 12) bytes per link · 1000 links per day ≈ 131 kilobytes per day, let's allocate extra space for indexes, new fields, etc., resulting in about 1 megabyte per record
 - Annual volume: 365 megabytes even without adding TTL.
- **Network:** It's important for us to understand how much information we transfer over the network to estimate resources, etc.
 - Input: ≈ 100-1000 B (a GET request with a hash) or (a PUT request with a link)
 - Output: ≈ 100-1000 B (HTTP 302 + headers)
 - Total about KB per second.

4 Architecture

4.1 What we want to see

You should design an ideal architecture you would like to see in a perfect scenario for your project. Here, we expect a solution diagram (ideally as an image with detailed annotations, but a detailed text description is also acceptable).

4.2 Example

Create a single service, as it contains relatively little logic; it will receive requests from users and return results. Inside the service, we need:

- **Data layer:**
 - PostgreSQL for storing links (transactional operations)
 - Redis for the most frequent links (redirect cache)
 - ClickHouse (analytics)
- **API:**
 - REST (public API)
 - gRPC (internal communication)

4.3 Technology justification

Technology	Reason for choice
PostgreSQL	JSONB support for metadata, ACID transactions
Redis	Latency < 1ms for read operations
gRPC	Efficient for streaming statistics

5 Implementation

5.1 What we want to see

You should describe what changes you made to the architecture due to time and budget constraints.

5.2 Example

- **Compromises:**
 - Instead of 3 regions — 1 VPS (Hetzner CX31)

- No ClickHouse (analytics in PostgreSQL), though you could skip analytics altogether - that's fine.
- No database replication (DB is a single point of failure)

6 Infrastructure

6.1 What we want to see

You should describe how you deployed and where your project is hosted.

6.2 Example

For the MVP, we chose a configuration that supports up to 5 RPS:

Parameter	Value
CPU	2 cores (Intel Xeon or AMD EPYC)
Memory	4 GB (1 GB for PostgreSQL, 1 GB for Redis)
Disk	20 GB SSD (10 GB for DB, 5 GB for logs)
Network	1 Gbit/s, traffic up to 1 TB/month

Table 1: Minimum VPS requirements

6.2.1 Hosting selection

Suitable providers for me:

- Hetzner Cloud (CX21: 2 cores, 4 GB RAM, \$4.75/month)
- DigitalOcean (Basic Droplet: \$6/month)
- Yandex Cloud (vCPU 2, 4 GB RAM — 1200 rubles/month)

I chose Hetzner as it's the cheapest.

6.2.2 OS deployment

Ubuntu 22.04 LTS

sudo apt update && sudo apt upgrade -y

sudo apt install -y docker-compose nginx certbot

6.2.3 Docker configuration

File docker-compose.yml:

version: '3.8'

services:

postgres:

image: postgres:14

environment:

POSTGRES_PASSWORD: "your_password"

volumes:

- pg_data:/var/lib/postgresql/data

redis:

image: redis:6-alpine

command: redis-server --save 60 1 --loglevel warning

volumes:

- redis_data:/data

volumes:

pg_data:

redis_data:

Startup:

```
sudo docker-compose up -d
```

6.2.4 Nginx setup

Config /etc/nginx/sites-available/shortener:

```
server {  
  
    listen 80;  
  
    server_name short.example.com;  
  
  
    location / {  
  
        proxy_pass <http://localhost:8000>;  
  
        proxy_set_header Host $host;  
  
    }  
}
```

Activation:

```
sudo ln -s /etc/nginx/sites-available/shortener /etc/nginx/sites-enabled/
```

```
sudo systemctl restart nginx
```

6.2.5 SSL via Certbot

```
sudo certbot --nginx -d short.example.com
```

6.2.6 Monitoring

Minimum set for tracking operation:

- Logs: journalctl -u nginx -f
- Load: htop
- Network: nload

Port	Service
5432	PostgreSQL
6379	Redis
443	Nginx (HTTPS)

Table 2: Open ports on the server

7 MVP

7.1 What we want to see

You should provide links to the code and a working MVP.

7.2 Submission Guidelines

You should submit:

- **Project repository:** a link to the source code of your project.
- **Working MVP:** a link to the deployed and working version of your project.
- **Short video demo:** a link to a short video demo of your project added to your repository. The video can be uploaded to YouTube or any other video hosting platform and should show how the project works.

The repository should contain everything required to review, run, and understand the project implementation.

The MVP should demonstrate the main functionality described in the project requirements.

The video demo should provide a quick overview of the project flow and help reviewers understand the main user scenario without having to run the project locally.

